Introduction to Python

Professor: Horacio Larreguy
TA: Eduardo Zago, Ernesto Dávila, Jose Luis Perez

ITAM Applied Research 2,
17/01/2024

General Perspective

Getting Started with Python

Lists, Locals, Functions and For Loops

Importing and Exporting Data using Pandas

Manipulating Data Frames using Pandas and Numpy

Graphing using Matplotlib

Downloading Python and Setting up the Environment

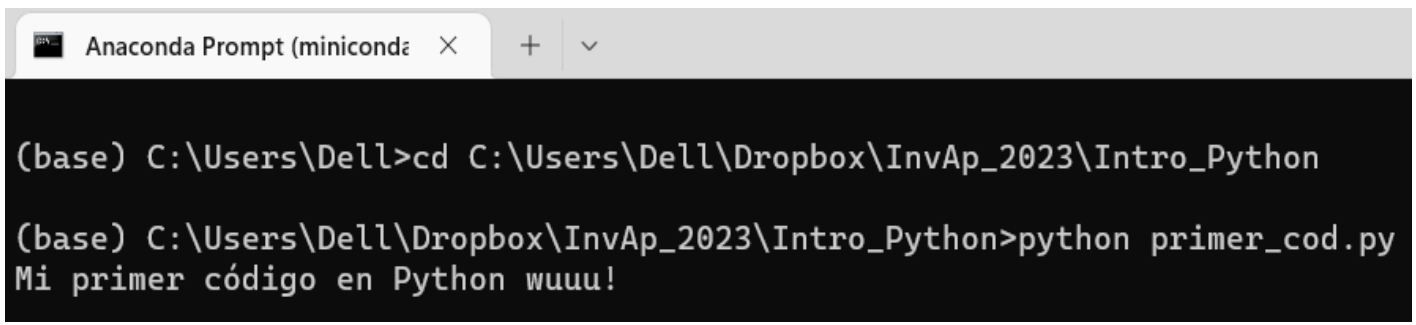
- To start using Python, we must follow a number of steps:
  1. Download Python.exe in your computer (notice it depends which operating system you have)
  2. Once that is installed, we must set up an Environment. The easiest one is downloading Mini Conda
  3. After the download is complete, run the installer and click through the setup steps leaving all the pre-selected installation defaults.
  4. Once complete, check that Miniconda was installed correctly by opening a Command Prompt (CMD or PowerShell for Windows) or a Terminal (for Mac) and entering the command 'conda list'

Python Code From the Terminal

- Once this is downloaded, go to *Anaconda Prompt (miniconda3)* and we can start using Python from the terminal.
- Notice the file in the DropBox Folder I sent you. We'll use the `print()` function:

```
# Changing the Directory (check yours):
cd \Dropbox\InvAp_2023\Intro_Python
# Call the py file
python primer_cod.py
```

Figure: Running Code from the Terminal (Output)



Jupyter Notebook

- Although running code from the terminal is useful, we will start using a more friendly code compiler: Jupyter Notebook

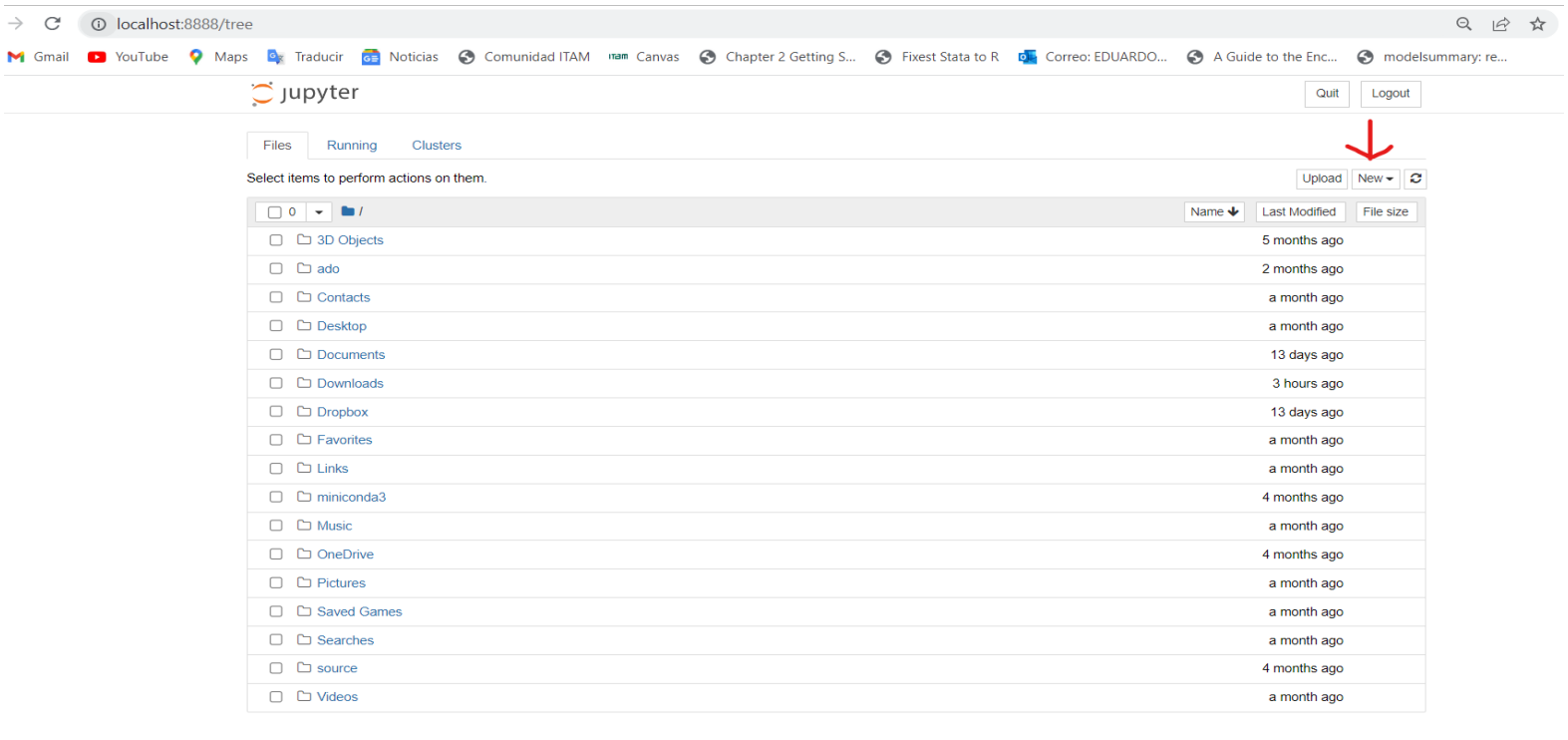
```
# Install Jupyter Notebook using pip:
pip install jupyter
```

```
# Run it in your terminal:
jupyter notebook
```

- You will get the following page browser:

Jupyter Notebook

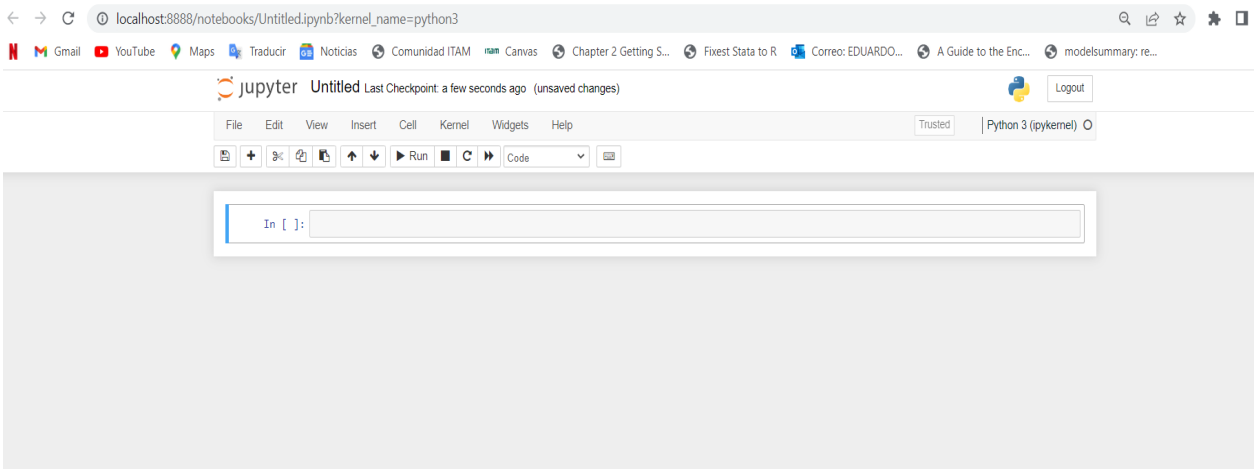
Figure: Jupyter Notebook's Main Menu



- To start writing code, we must do so in an .ipynb file. To create one select *New* > *Python 3 (ipykernel)* on your desired folder.

Jupyter Notebook

Figure: Python 3 Notebook



- Now we can start writing code and running it in a more friendly interface. For the whole tutorial, we will be using Python 3 Notebooks to run our code.
- More references: Visual Studio Code, CodeCademy: How to install Conda

Installing Packages

- Finally, to correctly set up the Environment, we need to download the necessary packages. In this case we will be using the function `!pip`, as follows:

```
# Installing packages using pip:
!pip install matplotlib
!pip install seaborn
```

- We must load them to the kernel to use their functions:

```
# Load the packages to the session:
import matplotlib.pyplot as plt
import seaborn as sns
```

pandas Package

- pandas is a fast, flexible and easy to use open source data analysis and manipulation tool.
- It is a DataFrame object for data manipulation with integrated indexing, tools for reading and writing data, flexible reshaping, merging, and intelligent data alignment.
- This is the go to package for data manipulation and cleaning in Python, such as dplyr is for R.
- To install it and load it use the following command:

```
# Install it
!pip install pandas

# Load the packages to the session:
import pandas as pd
```

NumPy Package

- NumPy is a Python library used for working with arrays.
- It also has functions for working in domain of linear algebra, fourier transform, and matrices.
- NumPy arrays are stored at one continuous place in memory unlike lists, so processes can access and manipulate them very efficiently.
- To install it and load it use the following command:

```
# Install it
!pip install numpy

# Load the packages to the session:
import numpy as np
```

Calculator and Locals

- Python can be used to perform operations as in a calculator.

```
# Sum
2 + 2
### Returns: 4
```

- Or save locals and perform operations on these variables:

```
# Define locals:
a = 5
b = 10
# Perform operation, save it in another local
c = a + b
# Print it
print(c)
### Returns: 15
```

Local Strings

- It will prove very useful to learn how to work with local strings, here are a few examples:

```
# Define local strings:
b1 = 'Hola'
b2 = 'Cómo estas?'

# Paste them and save it in another local
c1 = b1 + ' ' + b2
print(c1)
### Returns: 'Hola Cómo estas?'
```

```
# Useful when we want to work with relative paths:
base = '../../data/baseline/'
path_in = base + 'in/'
path_out = base + 'out/'

print(path_in, path_out)
### Returns: ../../data/baseline/in/ ../../data/baseline/out/
```

Local Strings

- To import data, we might want to do it in an automated way, thus using the function `f''` is very useful:

```
# Local variables
country1 = 'Argentina'
country2 = 'Mexico'

# Concatenated paths
base = f'../../data/baseline/{country1}/'
base2 = f'../../data/baseline/{country2}/'
path_in = base + 'in/'
path_out = base2 + 'out/'

print(path_in, path_out)
### Returns: ../../data/baseline/Argentina/in/
### ../../data/baseline/Mexico/out/
```

Creating Lists

- There are several objects that store information in Python, the most important ones being: data frames, dictionaries and lists.
- Lists help store 1-dimensional vectors of relevant information and are the easiest ones to loop on.
- There are several ways of defining a list:

```
# Define a list from scratch:
a = [1, 2, 3, 8, 9, 10, 20]

# Define an integer list for a certain range
b = range(3, 10) # From 3 to 9, increments of 1
c = range(3, 10, 2) # Increments of 2

# From a data frame column using pandas:
list_frame = df['ids'].tolist()
```

Manipulating Lists

- We can manipulate lists by adding elements, removing, concatenating with other lists, etc.

```
# Extracting specific elements:
a[0] # First element
a[-1] # Last element

# Subsetting lists
a[1:n] # from position 1 to n-1 [,)

# Getting the list's length
len(a)

# Appending new elements at the end of a list:
a.append('Hola')

# Removing elements by name from the list:
a.remove('Hola')
```

Functions

- The Python function syntaxis is the following:

```
def function1(input1, input2, ..., inputn):
      output1 = f(input1, ..., inputn)
      output2 = f(output1, input1, ..., inputn)
      .
      .
      .
      outputn = f(output1, ..., output(n-1), input1, ..., inputn)
 return output1, ..., outputn
```

- To call the function, you must provide the same amount of names as there are outputs:

```
df1, df2, ..., dfn = function1(input1, ..., inputn)
```

Function Examples:

```
# Prints and stores the output in a
def my_second_function(word1, word2):
    word = word1 + ' ' + word2
    print(word)
return word
a = my_second_function('Hola', 'cómo estas?')
# Input: Two Strings, Output: 1 String
```

```
def get_path(country):
    base = f'../../data/{country}/baseline/'
    path_in = base + 'in/'
    path_out = base + 'out/'
return base, path_in, path_out

base, path_in, path_out = get_path('Argentina')
print(base, path_in, path_out)

### Returns: ../../data/Argentina/baseline/
### ../../data/Argentina/baseline/in/
### ../../data/Argentina/baseline/out/
```

For Loops

- For loops in Python are relatively easier to understand than in R. Their syntaxis is much more friendly.

```
# Prints from 1 to 10
for i in range(1, 10):
    print(i)
```

```
# We could loop through lists and use functions inside the loop:
list_loop = ['Arg', 'Mex', 'Col']
for country in list_loop:
    base, path_in, path_out = get_path(country)
    print(base, path_in, path_out)

### Returns: ../../data/Arg/baseline/ ../../data/Arg/baseline/in/
### ../../data/Arg/baseline/out/
### ../../data/Mex/baseline/ ../../data/Mex/baseline/in/
### ../../data/Mex/baseline/out/
### ../../data/Col/baseline/ ../../data/Col/baseline/in/
### ../../data/Col/baseline/out/
```

Nested For Loops

- And more importantly, we could easily do nested for loops.

```
list_loop = ['Arg', 'Mex', 'Col']
type_of_data = ['baseline', 'treatment']

def get_path2(country, type_d):
    base = f'../../data/{country}/{type_d}/'
    path_in = base + 'in/'
    path_out = base + 'out/'
return base, path_in, path_out

for country in list_loop:
    for type_d in type_of_data:
        base, path_in, path_out = get_path2(country, type_d)
        print(base, path_in, path_out)
```

Figure: Output Loop 1

```
../../data/Arg/baseline/ ../../data/Arg/baseline/in/ ../../data/Arg/baseline/out/
../../data/Arg/treatment/ ../../data/Arg/treatment/in/ ../../data/Arg/treatment/out/
../../data/Mex/baseline/ ../../data/Mex/baseline/in/ ../../data/Mex/baseline/out/
../../data/Mex/treatment/ ../../data/Mex/treatment/in/ ../../data/Mex/treatment/out/
../../data/Col/baseline/ ../../data/Col/baseline/in/ ../../data/Col/baseline/out/
../../data/Col/treatment/ ../../data/Col/treatment/in/ ../../data/Col/treatment/out/
```

Nested For Loops Examples

```
# More Nested Loops:
for i in range(2, 4):
    # Printing inside the outer loop
    # Running inner loop from 1 to 10
    for j in range(1, 11):

        # Printing inside the inner loop
        print(i, "*", j, "=", i*j)
    # Printing inside the outer loop
    print()
```

Figure: Output Loop 2

```
2 * 1 = 2
2 * 2 = 4
2 * 3 = 6
2 * 4 = 8
2 * 5 = 10
2 * 6 = 12
2 * 7 = 14
2 * 8 = 16
2 * 9 = 18
2 * 10 = 20

3 * 1 = 3
3 * 2 = 6
3 * 3 = 9
3 * 4 = 12
3 * 5 = 15
3 * 6 = 18
3 * 7 = 21
3 * 8 = 24
3 * 9 = 27
3 * 10 = 30
```

Nested For Loops Examples

```
# Initialize list1 and list2 with some strings
list1 = ['I am ', 'You are ']
list2 = ['healthy', 'fine', 'geek']
list2_size = len(list2) # length list 2

# Running outer for loop to iterate through a list1.
for item in list1:
    print("start outer for loop ") # Printing outside inner loop
    i = 0 # Initialize counter i with 0
    # Running inner While loop to iterate through list2.
    while(i < list2_size):
        print(item, list2[i]) # Printing inside inner loop
        i = i+1 # Incrementing the value of i
    # Printing outside inner loop
    print("end for loop ")
### See the example code for the output
```

Figure: Output Loop 2

```
start outer for loop
I am  healthy
I am  fine
I am  geek
end for loop
start outer for loop
You are  healthy
You are  fine
You are  geek
end for loop
```

Importing Data

- Notice we have already imported the pandas package as pd, so to import data we can just use the functions available.
- We will start using relative paths with respect to where the Notebook is saved.

```
# Read excel using pandas
df1 = pd.read_excel('../data/twitter.xlsx')
# And we can visualize it
df1.head()
```

Figure: Pandas Data Frame

Out[9]:

	twitter_handle	text	possibly_sensitive	date	date_consulted	like_count	quote_count	reply_count	retweet_count	
0	BashirAhmaad	RT @francefootball: KARIM BENZEMA IS THE 2022 ...	False	2022-10-17T20:00:02.000Z	2022-10-17	0	0	0	65106	https://twitter.com
1	LandNoli	RT @wikileaks: Julian Assange speaking in 2011...	False	2022-10-17T04:22:35.000Z	2022-10-17	0	0	0	57220	https://twitter.cor
2	SphilileNxumalo	RT @michaelb4jordan: 3/3/23 #creed3 https://t....	True	2022-10-17T13:57:54.000Z	2022-10-17	0	0	0	47678	https://twitter.c
3	coolfrancess	RT @dipo_smart: The person he's proposing to i...	False	2022-10-17T06:43:38.000Z	2022-10-17	0	0	0	22621	https://twitter.cc
4	MbuyiseniNdlozi	RT @dipo_smart: The person he's proposing to i...	False	2022-10-17T07:22:18.000Z	2022-10-17	0	0	0	22621	https://twitter.cc

Exporting Data

- To export data we do this directly on a data frame.
- We can generate folders directly from Python using `os`

```
import os # for folder/path manipulation

# Function to create folders using os:
def create_folder(directory):
    """Create folder"""
    # Create target Directory if don't exist
    if not os.path.exists(directory):
        os.mkdir(directory)
        print("Directory ", directory, " created ")
    else:
        print("Directory ", directory, " already exists")

create_folder('../data/out/')
# Saving as parquet
df1.to_parquet('../data/out/twitter.parquet.gzip')
# Saving as Excel
df1.to_excel('../data/out/twitter.xlsx')
```

Looping through several Folders

- Sometimes the information required is in distinct folders with different names. To import and export this data in an efficient way we could use a loop that goes through the paths:

```
for i in range(1, 3):
    folder_out = f'../data/batch{i}/'
    create_folder(folder_out)
    for e in range(1, 4):
        df = pd.read_excel(f'../data/batch{i}/info{e}.xlsx')
        df.to_parquet(f'{folder_out}info{e}.parquet.gzip')
    print('Done')
```

Manipulating Data Frames

- Pandas package is the dplyr of Python. With this package we are going to be able to rename, select, drop, create columns, as well as filter our DFs.
- Here are a few important functions:

```
# Renaming:
df.rename({'twitter_handle': 'username',
    'possibly_sensitive': 'sensitive'},
    axis=1, inplace=True)

# Selecting variables:
df_info = df[['username', 'date', 'treatment']]

# Dropping variables:
df_date = df_info.drop(['username'], axis='columns')

# Creating new variables:
df['int_divided'] = df['total_interactions']/2
```

Pandas Functions

```
# Cool way to generate a list of variables with the same prefix, suffix
metrics = [col for col in df.columns if '_count' in col]

# Summing the columns (you can do .mean(), etc.)
df['total_ints_2'] = df[metrics].sum(axis=1)
# axis = 1 is axis = 'columns'

# If else in Python using numpy
df['greater_median'] = np.where(df['total_ints_2'] >
        df['total_ints_2'].median(), 1, 0)
```

Pandas Functions: Filtering Data

```
# Duplicates drop:
df_new = df.drop_duplicates('username')

# Filtering:
df_new = df[df['like_count'] > 0].reset_index(drop=True)

# Filtering (more than one condition):
df_new = df[(df['like_count'] > 110) &
    (df['quote_count'] > 10)].reset_index(drop=True)

# Filtering a column by the elements in a list
exclude_usernames = ['AfricaFactsZone', 'renoomokri', 'citizentvkenya']
df_new = df[(~df['username'].isin(exclude_usernames)) &
    (df['like_count'] > 110) &
    (df['quote_count'] > 10)].reset_index(drop = True)
```

Pandas Functions: Merging Data

```
# Generating a random data frame so we can merge it
import random

df.rename({'treatment': 'post_treatment'}, axis=1, inplace=True)
df_treat = df[['username']].reset_index(drop=True)
df_treat = df_treat.drop_duplicates()
df_treat['user_treatment'] = [random.randint(0, 1)
    for k in df_treat.index]

# Merging with the original data set:
df = df.merge(df_treat, on='username', how='left')
```

Pandas Functions: Apply

- To apply functions into our DFs and generate new variables, we must use the `apply()` function.
- While we are at it, let's also introduce a few techniques to clean strings.
- First we define the functions:

```
import re # Import regular expression package
def remove_url(text):
    return re.sub(r'https?:\S*','',text)

# Remove mentions and hashtags
def remove_mentions_and_tags(text):
    text = re.sub(r'@\S*','',text)
    return re.sub(r'#\S*','',text)
```

Pandas Functions: Apply

- We leverage the `re` package to detect and substitute any string that matches with the one specified.
- Notice we cannot apply that function directly to a DF column, as in R:

```
string1 = 'Hola, esta es la página de
'       The NYT https://www.nytimes.com/international/'
remove_url(string1)
### Output: 'Hola, esta es la página de The NYT '

string2 = 'Y el username es @NYT y su HT es #NYT'
remove_mentions_and_tags(string2)
### Output: 'Y el username es  y su HT es '

df['text_clean'] = remove_url(df['text'])
### TypeError: expected string or bytes-like object
```

Pandas Functions: Apply

- We must use the following sintaxis:

```
df['text_clean'] = df['text'].apply(remove_url)
df['text_clean'] = df['text_clean'].apply(remove_mentions_and_tags)
```

- We can also use the apply function, along with the `lambda` one to modify all variables.

```
df_t = df[['reply_count', 'like_count',
        'quote_count']].apply(lambda x: np.square(x))

df = df.assign(sum_interactions = lambda x: (x['reply_count'] +
        x['like_count'] + x['quote_count']))
```

Pandas Functions: Summarizing Data

```
# Getting the mean of a certain variable:
df['retweet_count'].mean()
### Returns: 2848.027932960894

# Or do a table with these locals:
data_summ = {'Type of Interaction': ['RTs', 'Likes', 'Quotes', 'Replies'],
'Mean': [df['retweet_count'].mean(), df['like_count'].mean(),
        df['quote_count'].mean(), df['reply_count'].mean()],
"SD": [df['retweet_count'].std(), df['like_count'].std(),
        df['quote_count'].std(), df['reply_count'].std()],
"Min.": [df['retweet_count'].min(), df['like_count'].min(),
        df['quote_count'].min(), df['reply_count'].min()],
"Max.": [df['retweet_count'].max(), df['like_count'].max(),
        df['quote_count'].max(), df['reply_count'].max()]}

data_summ = pd.DataFrame(data_summ)
print(data_summ.to_latex()) # To LaTeX
```

	Type of Interaction	Mean	SD	Min.	Max.
0	RTs	2848.027933	7966.844376	29	65106
1	Likes	391.955307	883.994959	0	7166
2	Quotes	8.240223	25.657716	0	269
3	Replies	52.664804	139.827058	0	1196

Table: Summary Statistics

Pandas Functions: Summarizing Data

```python
# We can get grouped summaries:
metrics = ['retweet_count', 'like_count', 'quote_count', 'reply_count']
grouped_summ = df[['post_treatment'] +
        metrics].groupby('post_treatment').mean()

# And export to LaTeX:
print(grouped_summ.to_latex())
```

post_treatment	retweet_count	like_count	quote_count	reply_count
0	2237.584615	840.646154	11.707692	112.569231
1	3196.087719	136.122807	6.263158	18.508772

Table: Grouped Summary Statistics

Example using For Loops and pandas

- Look at the files located in the data folder: notice we have two folders containing three Excel files each.
- We would like to loop through these files and append them one in top of the other, this is what we call aggregating data into one file.
- We would also like to add columns to each file while appending, to do this we can use the pandas package.

```
# Example with one file:
example = pd.read_excel('../data/batch1/info1.xlsx')

# Add the batch
example['batch'] = 1

#Add the info:
example['info'] = 1
```

Example using For Loops and pandas

- We can build a function to use it in a loop:

```
def read_info(batch_num, info_num):
    df = pd.read_excel(f'../data/batch{batch_num}/info{info_num}.xlsx')
    df['batch'] = batch_num
    df['info'] = info_num
return df

# Let's see the output:
example = read_info(1,1)
example
```

Figure: Data Frame Example

	id	mujer	edad	batch	info
0	1001	1	22	1	1
1	1002	1	12	1	1
2	1003	0	23	1	1
3	1004	1	43	1	1

Example using For Loops and pandas

- Finally we loop through the paths:

```
df_final = pd.DataFrame() # Initialize final df
for i in range(1, 3):
    df_int = pd.DataFrame() # Initialize intermediate df for the inner loop
    for e in range(1, 4):
        df = read_info(i, e)
        df_int = pd.concat([df_int, df])
    df_final = pd.concat([df_final, df_int])

df_final = df_final.reset_index(drop = True)
df_final
```

- And we get the following output:

Example using For Loops and pandas

Figure: Data Frame Example

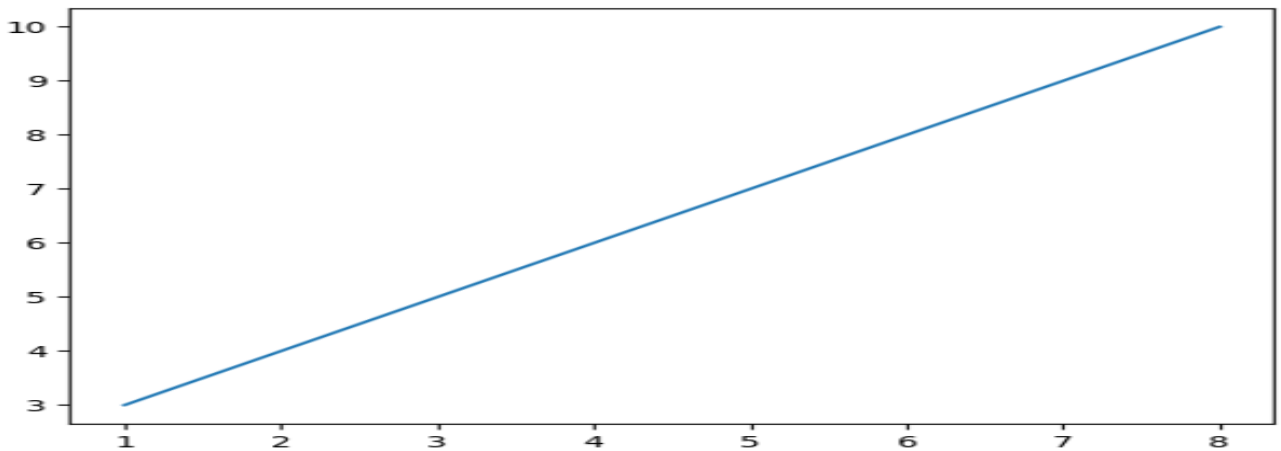
	id	mujer	edad	batch	info
0	1001	1	22	1	1
1	1002	1	12	1	1
2	1003	0	23	1	1
3	1004	1	43	1	1
4	1005	0	10	1	2
5	1006	1	15	1	2
6	1007	0	22	1	2
7	1008	1	39	1	2
8	1010	0	17	1	3
9	1011	0	22	1	3
10	1012	1	33	1	3
11	1013	1	15	1	3

Matplotlib: Line

- The go to package for graphing in Python is Matplotlib.pyplot

```
xpoints = np.array([1, 8])
ypoints = np.array([3, 10])

plt.plot(xpoints, ypoints)
plt.show()
```

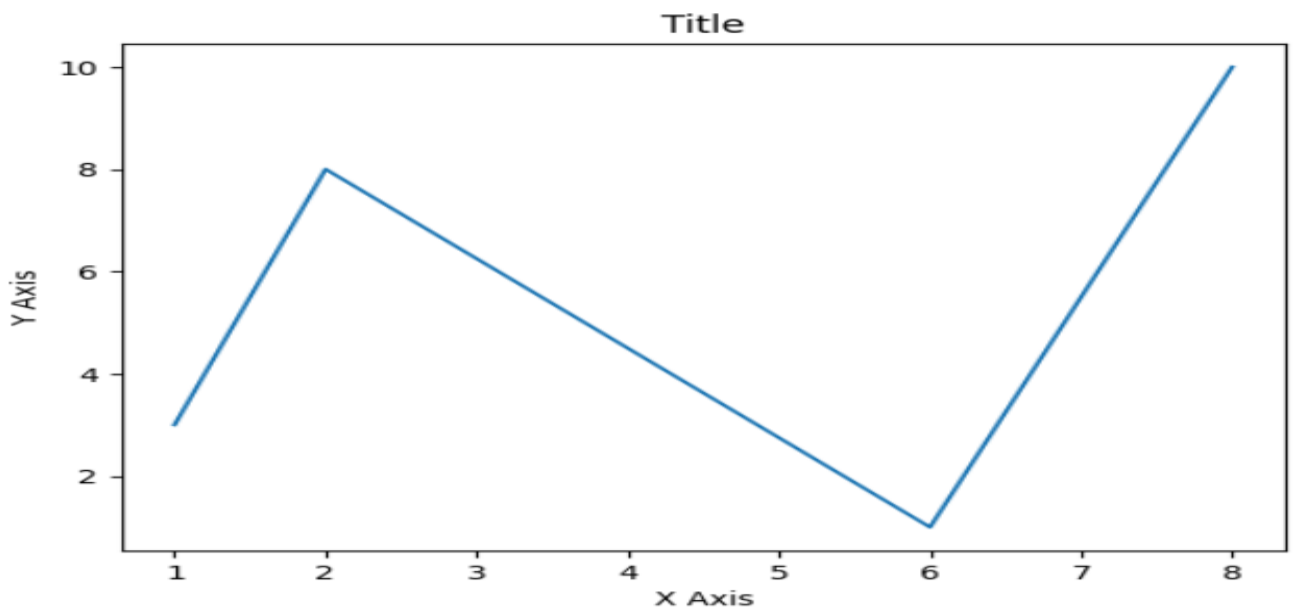
Figure: Line



Matplotlib: Titles and Axis

```
xpoints = np.array([1, 2, 6, 8])
ypoints = np.array([3, 8, 1, 10])
plt.plot(xpoints, ypoints)
plt.title('Title')
plt.xlabel("X Axis")
plt.ylabel("Y Axis")
plt.show()
```

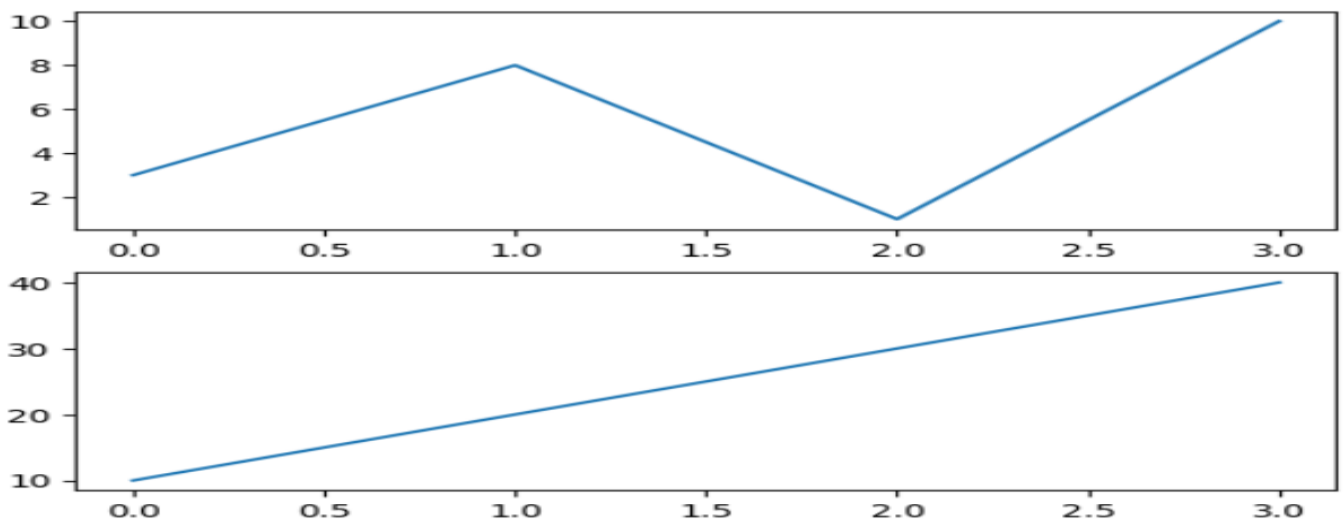
Figure: Titles and axis



Matplotlib: Subplots

```
#plot 1:
x = np.array([0, 1, 2, 3])
y = np.array([3, 8, 1, 10])
plt.subplot(2, 1, 1)
plt.plot(x,y)

#plot 2:
x = np.array([0, 1, 2, 3])
y = np.array([10, 20, 30, 40])
plt.subplot(2, 1, 2)
plt.plot(x,y)
plt.show()
```

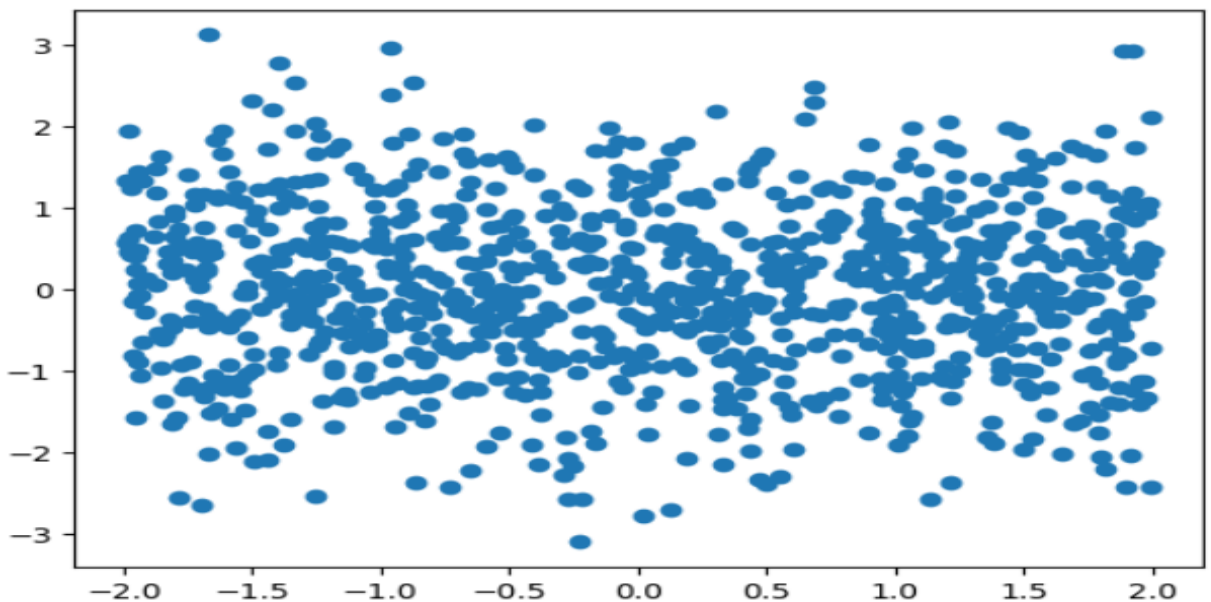
Figure: Subplots



Matplotlib: Scatter Plots

```
# We generate two vectors of random variables (uniform and normally distributed)
from numpy import random
y = random.normal(loc=0, scale=1, size=(1000))
x = random.uniform(-2, 2, size=(1000))

plt.scatter(x, y)
plt.show()
```

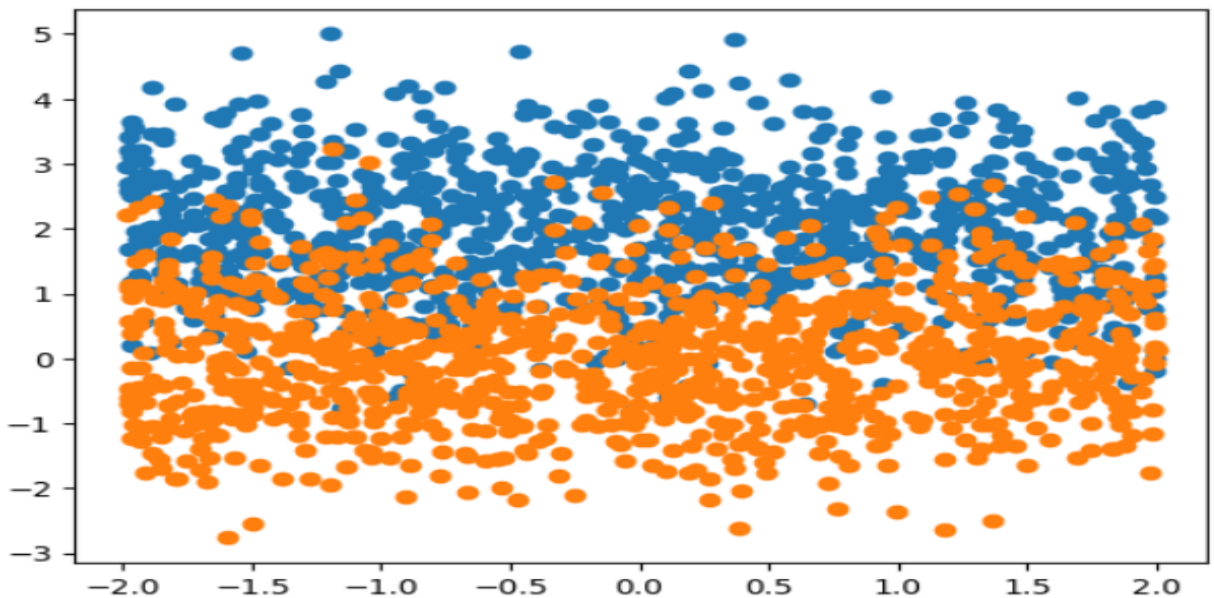
Figure: Scatter Plot



Matplotlib: Scatter Plots

```
# We can also graph two data points that come from different distributions
y = random.normal(loc=0, scale=1, size=(1000))
x = random.uniform(-2, 2, size=(1000))
y2 = random.normal(2, 1, size = (1000))

plt.scatter(x,y2)
plt.scatter(x, y)
plt.show()
```

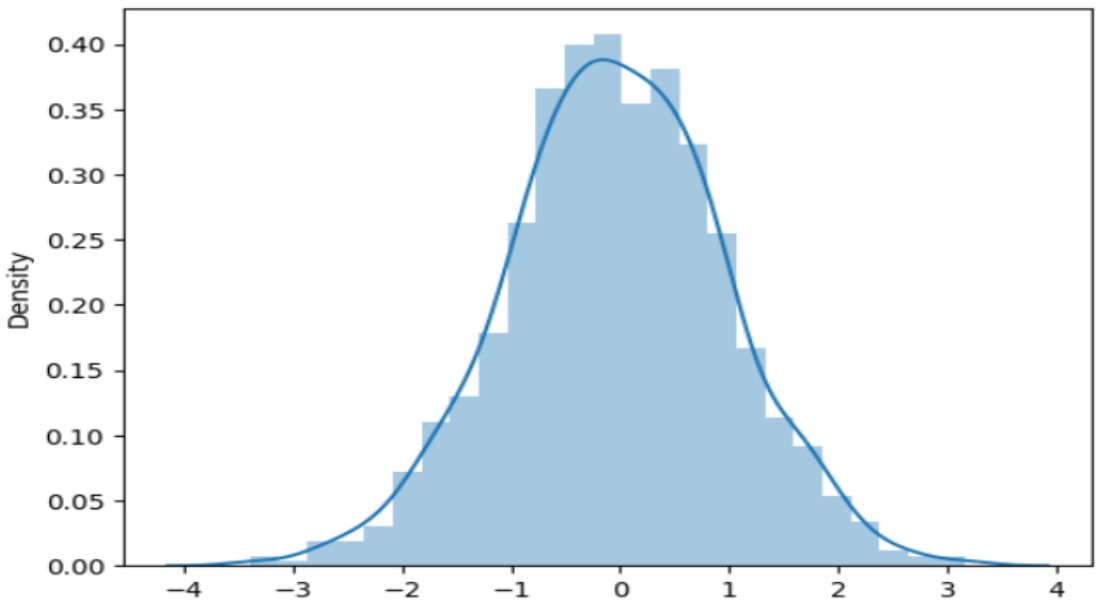
Figure: Mixed Scatter Plot



Matplotlib: Distributions

```
y = random.normal(loc=0, scale=1, size=(1000))
x = random.uniform(-2, 2, size=(1000))
y2 = random.normal(2, 1, size = (1000))

import seaborn as sns

sns.distplot(y, hist=True)
```

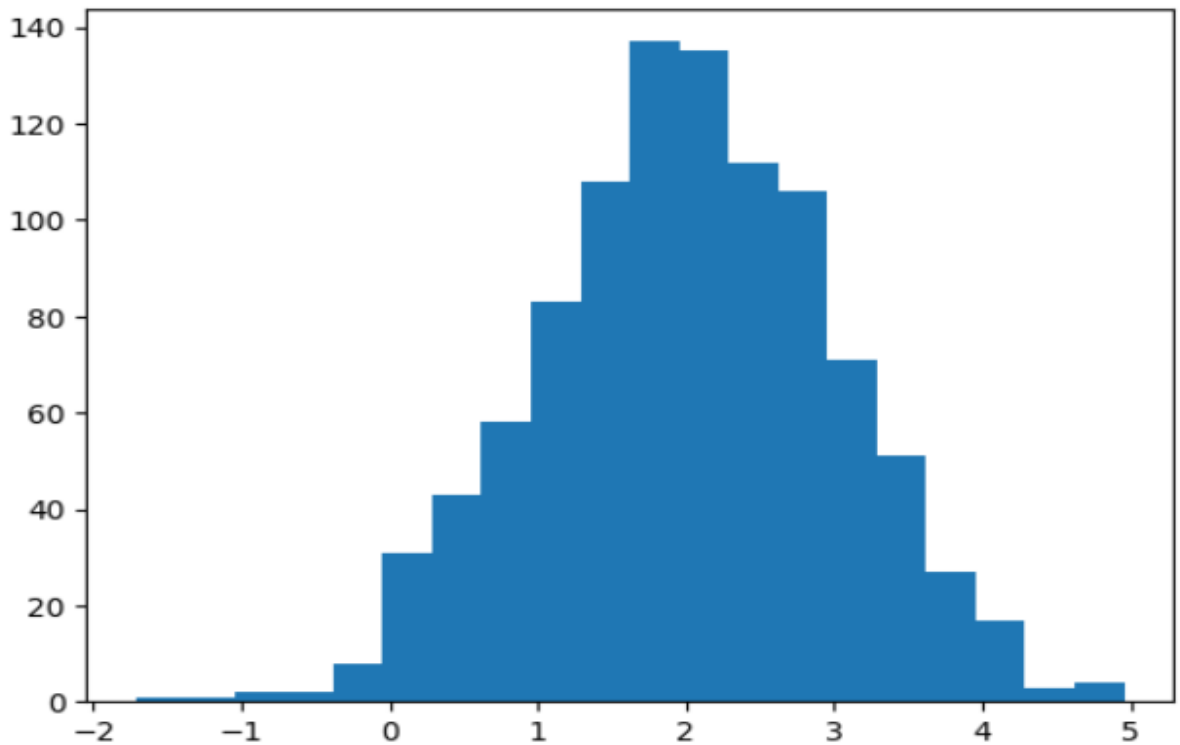
Figure: Normal Distibution



Matplotlib: Histogram

```
y2 = random.normal(2, 1, size = (1000))
plt.hist(y2, bins = 20)
plt.show()
```

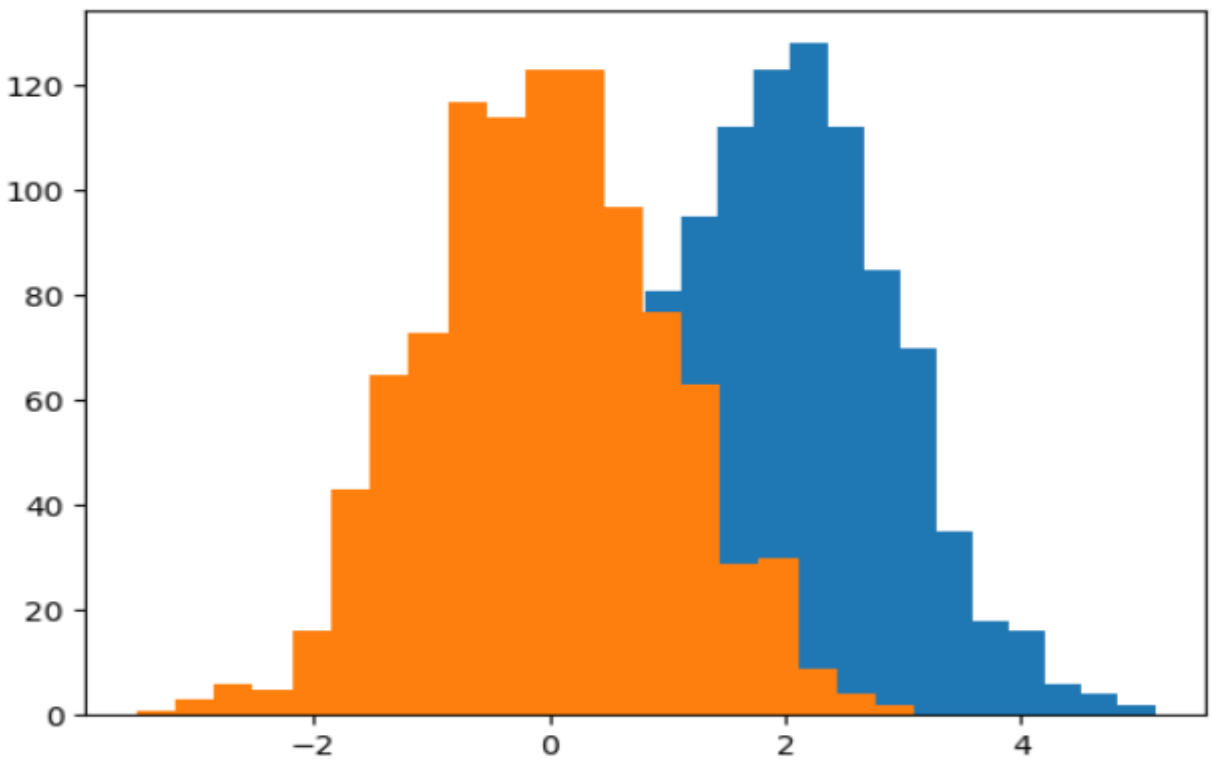
Figure: Histogram



Matplotlib: Histogram

```
y = random.normal(loc=0, scale=1, size=(1000))
y2 = random.normal(2, 1, size = (1000))

plt.hist(y2, bins = 20)
plt.hist(y, bins = 20)
plt.show()
```

Figure: Histogram



Matplotlib: Iris Example

- One important aspect of visualization is preparing the data to graph it.
- At the moment we have just seen easy examples using vectors.
- Let us try to build a scatter plot with more complicated data: the Iris data set.

```
from sklearn.datasets import load_iris
iris = load_iris()

iris_df = pd.DataFrame(data=iris.data, columns=iris.feature_names)

# Add the target variable to the DataFrame
iris_df['class'] = iris.target
```

Matplotlib: Iris Example

Figure: Iris Data Set

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)	class
0	5.1	3.5	1.4	0.2	0
1	4.9	3.0	1.4	0.2	0
2	4.7	3.2	1.3	0.2	0
3	4.6	3.1	1.5	0.2	0
4	5.0	3.6	1.4	0.2	0
...	...	...	...	...	...
145	6.7	3.0	5.2	2.3	2
146	6.3	2.5	5.0	1.9	2
147	6.5	3.0	5.2	2.0	2
148	6.2	3.4	5.4	2.3	2
149	5.9	3.0	5.1	1.8	2

150 rows × 5 columns

Matplotlib: Iris Example

- Let us generate a variable that contains the name of each class.
- This also help us introduce a way to generate variables with several conditions and classes (such as `case_when()` in R).

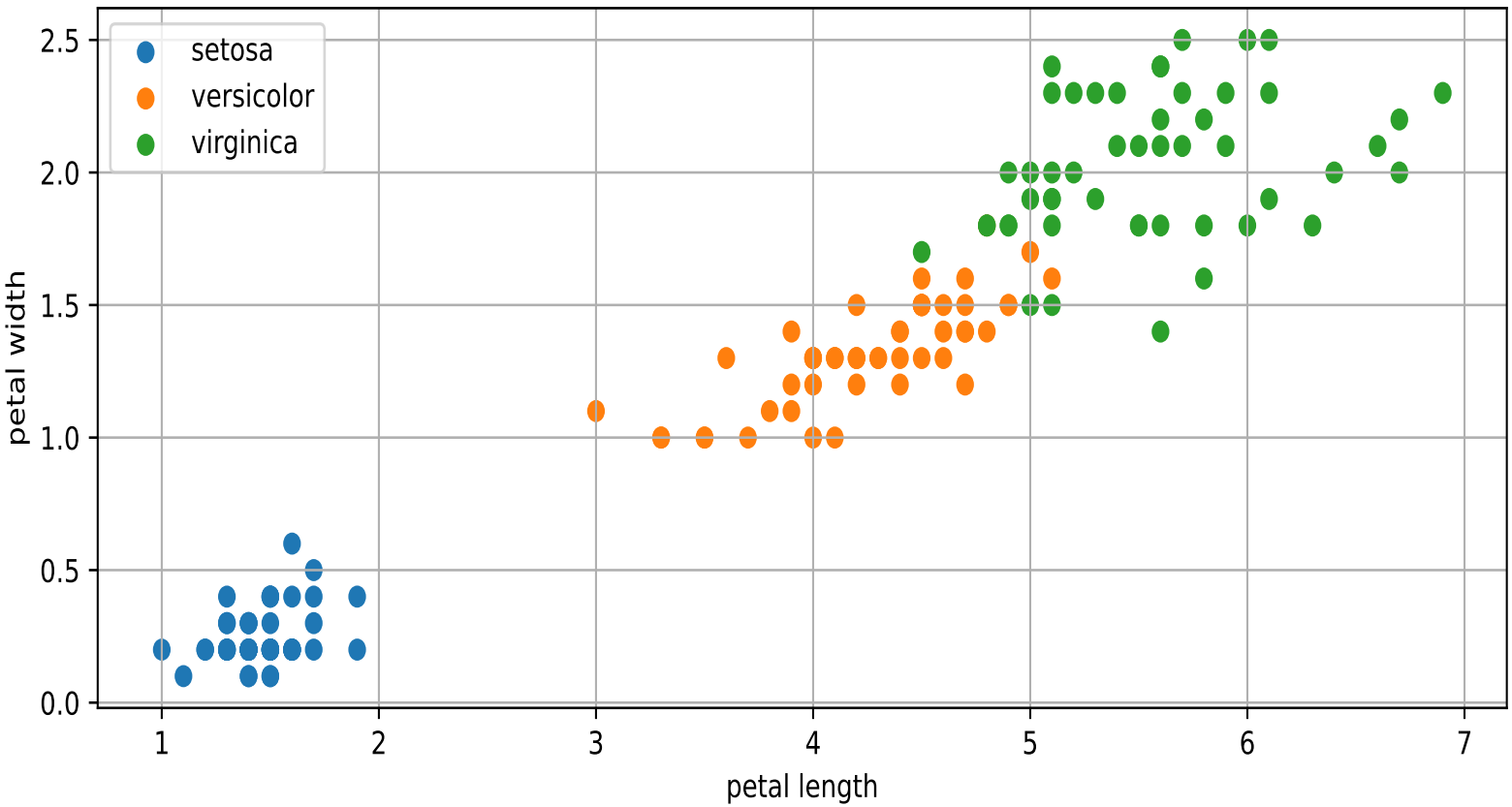
```
# Define conditions
conditions = [
    (iris_df['class'] == 0),
    (iris_df['class'] == 1),
    (iris_df['class'] == 2)
]

# Define corresponding values for each condition
values = ['setosa', 'versicolor', 'virginica']

# Use numpy's select to apply conditions
iris_df['name'] = np.select(conditions, values, default='Undefined')
```

Matplotlib: Iris Example

- Finally, we can graph it using all what we have seen:

```
plt.figure(figsize=(10, 4))
plt.scatter(iris_df[iris_df['class'] == 0][['petal length (cm)']],
            iris_df[iris_df['class'] == 0][['petal width (cm)']],
            label='setosa')
plt.scatter(iris_df[iris_df['class'] == 1][['petal length (cm)']],
            iris_df[iris_df['class'] == 1][['petal width (cm)']],
            label='versicolor')
plt.scatter(iris_df[iris_df['class'] == 2][['petal length (cm)']],
            iris_df[iris_df['class'] == 2][['petal width (cm)']],
            label='virginica')
plt.legend()
plt.grid(True)
plt.xlabel('petal length')
plt.ylabel('petal width')
plt.savefig("iris.pdf")
```

Figure: Scatter Plot Iris



QUESTIONS